다듬(DaDeum) 소개서

다듬(DaDeum) 소개서

거친 생각에서, 단단한 문장으로. 한국어 맞춤법·띄어쓰기·조사·문법을 **내 컴퓨터 안에서** 교정하는 도구

> 이 문서는 발표 질문에 답할 수 있을 만큼의 핵심만 담았습니다. 전체 설명은 `다듬-완전정복.pdf` 에 있습니다. 수치는 2026-10-11에 저장소를 받아 `python -m eval.pipeline` 으로 직접 잰 값입니다.

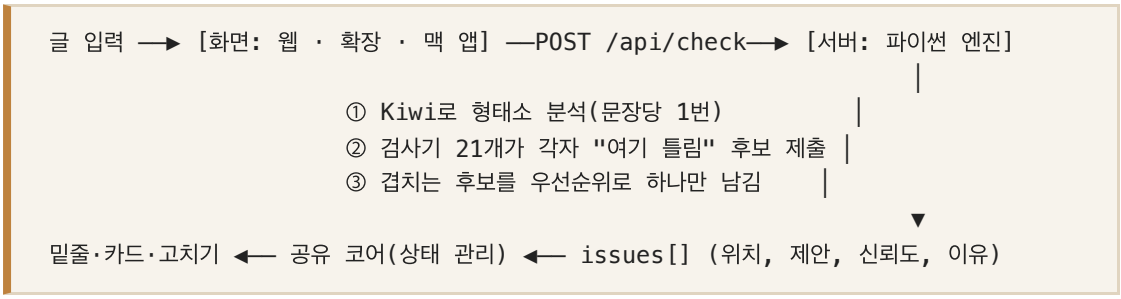
1. 한 문단 소개

글을 쓰면 틀린 곳에 밑줄이 그어지고, 카드에서 **무엇을·왜** 고치는지 읽고, 클릭 한 번에 고칩니다. 웹, 크롬 확장, macOS 앱 세 가지 모습이 **같은 엔진 하나**를 씁니다. 외부 AI·계정·과금이 없고 글이 기기를 떠나지 않습니다.

2. 왜 만들었나 / 무엇이 다른가

질문	답
해결하는 문제	맞춤법 검사기는 많지만 **멀쩡한 글을 틀리게 고치는** 일이 잦아 사용자가 한 번에 끈다
다듬의 태도	**놓치는 것보다 틀리게 고치는 것을 훨씬 무겁게 본다.** 확신이 없으면 고치지 않고 점선으로만 표시
차별점 1	완전 로컬: 외부 API·계정·과금 없음, 글이 밖으로 나가지 않음
차별점 2	모델을 직접 학습 (판별 모델 5개 + 언어모델 1개). 학습 알고리즘(scikit-learn)과 코퍼스(위키백과·nsmc)만 빌림
차별점 3	숫자를 정직하게 보고: 잘 나온 숫자만이 아니라 **처음 보는 문장** 성적을 따로 공개

3. 어떻게 동작하나 (그림 하나)



- 신뢰도 **high/medium** = 실선 밑줄, 클릭 한 번에 적용. **low** = 점선, 설명만(자동 적용 안 함).

4. 핵심 로직 7가지

① 받침은 산수로 구한다

한글 11,172자는 `(초성×21+중성)×28+종성` 순서로 번호가 매겨져 있어서 **`(글자번호−44032) % 28`이 받침 번호**(0이면 없음)입니다. `학교을`의 `교`는 0 → `를`이어야 함. 조사·율/률이 모두 이 한 줄에 기댑니다.

② 틀린 조사는 형태소 분석기도 못 알아본다

`학교을`의 `을`을 Kiwi는 조사가 아니라 명사(NNG)로 읽습니다. 그래서 "조사 태그를 찾는" 대신 **명사 바로 다음 글자**를 직접 검사합니다.

③ 규칙은 "예외 처리"가 대부분이다

사전 규칙 474개는 앞뒤 문맥을 정규식에 넣어 구분합니다. `카드로 결재`→`결제`, `서류를 결재`는 그대로. 규칙마다 정문(맞는 글)에서 발동한 것을 전수로 읽고 좁혔습니다(인명 `성희안`을 `성희한`으로 고치려던 규칙 9개를 찾아 막음).

④ 학습 모델은 "정문에서 문제와 정답을 동시에" 만든다

위키 문장에서 `되/돼` 자리를 `▒`로 가리면 **원문에 쓰인 글자가 곧 정답**입니다. 틀린 문장을 모을 필요가 없습니다. 글자 2~4조각을 세어(TF-IDF) 조각마다 가중치를 더하는 로지스틱 회귀입니다. **손계산이 모델 출력과 일치**하도록 직접 검증했습니다.

⑤ 문턱은 사람이 아니라 측정이 정한다

"정문을 잘못 건드리는 비율 ≤ 0.5%"를 지키는 **가장 낮은 확률 문턱**을 자동 선택합니다. 못 맞추면 그 모델은 아예 싣지 않습니다. 문법으로 답이 정해지는 자리(`되`+`고`, `든`+명사 등)는 확률을 묻지 않고 규칙이 먼저 확정합니다.

⑥ 겹치는 후보는 "제안 있는 쪽 → 우선순위"로 정리

고칠 말이 있는 지적이 없는 지적(이상탐지·문체)을 이깁니다. 같은 조건이면 규칙 6 > 사전 5 > 모델 4 > … 순.

⑦ 측정 도구도 의심한다

- 합성 오류로 잰 점수(93.8%)와 사람이 쓴 오류 점수(약 15%)가 달랐던 일 이후 **실사용 오류셋을 따로** 만들었다.
- 표본이 20건 미만이면 오탐률을 숫자로 내지 않는다("0%"가 아니라 "못 쟀다").
- 하한(후퇴 금지선)은 통과시키려고 내리지 않는다.

5. 숫자 (직접 돌린 값)

지표	값	의미
정밀도	**99.21%**	고친 것 중 맞은 비율 (위키 정문 9,765 + 합성 오류 3,871)
정문 오탐률 / 과교정률	**0.00%**	맞는 글을 건드린 비율 (하한 ≤ 0.2%)
규칙층 / 혼동쌍 재현율	97.9% / 99.2%	합성 오류 기준
실사용 오류 자동교정	dev 74% · holdout 72% · **holdout2(처음 본 문장) 67%**	사람이 실제로 쓴 오류
단위 테스트	1,020개 통과	
속도	문장 약 수 ms (서버 가동 후)	

> 발표 때는 **67%를 먼저** 말하세요. 74%는 규칙을 만들며 계속 본 평가셋이라 과적합이 섞여 있습니다. (README는 holdout2를 63%로 적고 있고, 제 환경 실측은 67%입니다. 차이의 원인은 확인하지 못했으므로 "약 60%대 중후반"이라고 말하는 것이 안전합니다.)

6. 구성

구성요소	내용
엔진	Python · FastAPI, 검사기 21개, 출처 58개
모델	혼동쌍 4개 가동(안/않·되/돼·던/든·로서/로써) + 경계 판별기 + 형태소 언어모델 + 어절 사전(181만) + 이상탐지
화면	웹(바닐라 JS), 크롬 확장(MV3), macOS 앱(Swift, 접근성 API)
공유 코어	밑줄·고치기·무시·되돌리기 상태관리를 JS와 Swift에서 같은 테스트로 일치
품질	한 명령(`eval.pipeline`)이 단위 테스트·QA 불변식·메트릭 하한을 순서대로 검증

7. 예상 질문과 답변

Q1. 맞춤법 검사기는 이미 많은데 뭐가 다른가요? 오탐(맞는 글을 틀리게 고침)을 가장 무겁게 봅니다. 확신이 없으면 고치지 않고 점선으로만 표시하고, 자동 교정은 규칙이 확정한 자리에서만 합니다. 그리고 전부 로컬입니다.

Q2. AI(ChatGPT 같은)를 쓰나요? 아니요. 모델은 이 저장소에서 처음부터 학습한 작은 판별 모델들입니다. 외부 AI API를 부르지 않습니다. (초기에는 Gemini 층이 있었지만 제거했습니다.)

Q3. 직접 만든 모델이라는 게 정확히 무슨 뜻인가요? 학습 데이터 수집, 문제·정답 생성, 학습, 문턱 선택, 파일 변환이 모두 이 저장소 코드입니다. 빌린 것은 학습 알고리즘(scikit-learn)과 코퍼스 원문뿐입니다. **단, 형태소 분석은 외부 라이브러리 Kiwi를 씁니다.** 규칙 검사기 대부분이 그 위에서 돕니다.

Q4. 정확도가 얼마인가요? 정문에서 고친 것 중 맞은 비율(정밀도)은 99.2%, 맞는 글을 건드린 비율은 0.0%입니다. 다만 **사람이 실제로 쓴 오류 중 자동으로 고치는 비율은 처음 본 문장 기준 약 67%**입니다. 나머지는 점선으로 표시하거나 놓칩니다.

Q5. 왜 100%에 못 가나요? 세 가지 원인이 있습니다. ① 의미로만 갈리는 것(`칼로서/칼로써`)은 글자 조각 모델이 약합니다(로서/로써 모델 단독 재현율 20%). ② 글자 하나 빠짐·중복 같은 오타는 단어를 알아야 잡습니다(퍼즈 검출 3~5%). ③ 맞춤법이 허용하는 표기가 많아서 일부러 침묵합니다.

Q6. 모델이 어떻게 판단하나요? `되/돼` 자리를 가리고 앞뒤 12글자를 봅니다. 글자 2~4조각마다 학습된 가중치를 더해 확률을 냅니다. 예: `잘 ▒면`에서 `▒면` 조각이 `되` 쪽으로 가장 크게 밉니다(확률 0.87).

Q7. 학습 데이터는 어디서 났나요? 라이선스는? 한국어 위키백과(CC BY-SA 4.0)와 nsmc 영화평(CC0)입니다. `NOTICE` 파일에 출처를 적었고, 모델은 문장이 아니라 통계 가중치만 담지만 CC BY-SA 파생물로 표기했습니다.

Q8. 개인정보는 어떻게 되나요? 서버가 `localhost`에서 돌고 글이 외부로 전송되지 않습니다. 비밀번호 필드와 보안 입력 모드는 읽지 않습니다.

Q9. 틀리게 고치면 어떻게 하나요? 되돌리기(최대 50단계), 이 교정 무시, "이 단어는 항상 허용"(개인 사전)이 있습니다. 과교정을 신고하면 그 문장이 그대로 회귀 테스트가 됩니다.

Q10. 느리지 않나요? 엔진은 문장 하나에 수 ms입니다(서버를 처음 띄운 직후 첫 요청만 사전 적재로 수 초). 입력 중에는 0.25~0.9초 디바운스로 검사합니다.

Q11. 긴 글도 되나요? 서버는 2만 자까지 받고, 4천 자씩 문장 경계에서 잘라 돌립니다. 웹앱 화면은 현재 5천 자에서 막습니다.

Q12. 구어(카톡·댓글)는요? 재현율은 95%지만, 구어 코퍼스의 "정문"에도 실제 띄어쓰기 오류가 많아서 정밀도는 재지 않습니다. 대신 사람이 보는 지적 빈도(두 문장 반에 하나꼴, 0.397)를 하한으로 관리합니다.

Q13. 한글 입력 중에 방해하지 않나요? 조합 중(`compositionstart` ~ `compositionend`)에는 아무것도 건드리지 않습니다. 밑줄 오버레이는 클릭·타이핑을 가로채지 않습니다.

Q14. 어떻게 품질을 지키나요? 한 명령이 단위 테스트 1,020개, QA 불변식(고쳐서 나빠지지 않음·두 번 고쳐도 같음 등), 정문 오탐 하한을 순서대로 검증합니다. 고친 버그는 회귀 목록에 영구 등록합니다.

Q15. 가장 큰 약점은? ① 사람이 쓴 오류의 약 3분의 1은 자동으로 못 고칩니다. ② 크롬 확장은 실제 Chrome에서의 재확인이 필요하다고 문서에 적혀 있습니다. ③ 평가의 주석자가 한 명이고 표본이 약 500문장입니다. ④ 이상탐지 통계 파일이 저장소에 없어 새 환경에서는 그 층이 꺼집니다.

Q16. 앞으로 무엇을 하나요? 처음 보는 문장에서의 재현율을 올리는 것이 기준입니다. 경계 판별에는 글자 조각 방식의 한계가 확인되어 다른 종류의 모델이 필요하다고 코드에 적혀 있습니다.

8. 30초 발표 요약

> "다듬은 한국어 맞춤법 교정기입니다. 다른 도구와 다른 점은 **틀리게 고치는 것을 가장 무겁게 본다**는 것입니다. 확신이 있을 때만 자동으로 고치고, 아니면 점선으로 알려만 줍니다. 엔진은 내 컴퓨터 안에서만 돌고, 규칙 검사기 21개와 직접 학습한 판별 모델

> 들이 같은 후보 목록에 의견을 내면 우선순위로 정리합니다. 정문에서 고친 것의 99%가 맞고 맞는 글은 건드리지 않으며, 사람이 실제로 쓴 오류는 처음 보는 문장 기준 약 67%를 자동으로 고칩니다. 못 잡는 것과 약점도 문서에 그대로 공개합니다.”